

Robust PCA для background subtraction

Principal Component Pursuit: фон как low-rank, движение как sparse

$$M = [\text{vec}(I_1), \dots, \text{vec}(I_T)] \in \mathbb{R}^{p \times T}, \quad M = L + S$$

Проект по теме

Сравнение выпуклого RPCA-ADMM и быстрых approximate/baseline методов на видео.

Результат проекта

5 методов · 3 видео · 180 запусков · runtime/memory/quality + visual decomposition grids.

Постановка задачи: что хотим оптимизировать

Цель — разделить видео на неподвижный фон и движущиеся объекты

Что дано

Последовательность кадров видео. Каждый кадр превращаем в вектор, затем собираем матрицу наблюдений M .

Что ищем

L — низкоранговый фон, стабильный во времени. S — разреженные движения, выбросы и аномалии.

Зачем

Background subtraction для видеонаблюдения, предобработки видео, детекции движения и аномалий.

Гипотеза проекта

- ADMM-based RPCA должен давать более точное low-rank + sparse разложение, но хуже масштабироваться по времени и памяти.
- Windowed/approximate методы должны давать более практичный компромисс для видео: быстрее, меньше память, качество немного хуже.
- Baseline PCA/randomized PCA нужны как нижняя граница по времени, но они не являются robust-методами.

Критерии сравнения

качество разложения · runtime · peak memory · масштабируемость при росте frames/resolution · визуальная интерпретация foreground/background

Как видео превращается в матрицу

Рост числа пикселей и кадров напрямую увеличивает стоимость SVD



$$M = [\text{vec}(I_1), \dots, \text{vec}(I_T)] \in \mathbb{R}^{p \times T}, \quad M = L + S$$

- В нашем pipeline: grayscale, resize до 80×60 / 160×120 / 320×240.
- Для каждого видео берутся первые 50 / 100 / 200 / 400 кадров.
- Матрица имеет размер pixels × frames: например, 160×120 и 200 кадров дают $M \in \mathbb{R}^{19200 \times 200}$.
- Если $p = H \cdot W$ или T растут, SVD-based методы резко дорожают.

Интуиция low-rank

Фон меняется мало: разные кадры похожи друг на друга, поэтому columns of M лежат около низкоразмерного подпространства.

Интуиция sparse

Движение занимает небольшую часть пикселей в каждом кадре.

RPCA / Principal Component Pursuit

Невыпуклая идея rank+sparsity заменяется выпуклой релаксацией

Идеальная, но трудная постановка

$$\min_{L, S} \text{rank}(L) + \lambda \|S\|_0 \quad \text{s. t.} \quad L + S = M$$

Выпуклая релаксация PCP

$$\min_{L, S} \|L\|_* + \lambda \|S\|_1 \quad \text{s. t.} \quad L + S = M$$

$$\|L\|_* = \sum_i \sigma_i(L), \quad \|S\|_1 = \sum_{i,j} |S_{ij}|$$

L =
background

S = foreground /
anomalies

M =
observations

Почему это convex optimization

Ядерная норма — выпуклая оболочка rank на спектральном шаре; ℓ_1 -норма — стандартная выпуклая релаксация sparsity.

Почему работает для background subtraction

Стационарный фон хорошо аппроксимируется low-rank, а движущиеся объекты попадают в sparse-компоненту.

Что оптимизируем на практике

Ищем баланс: минимальный residual и адекватные foreground masks без чрезмерного runtime и memory.

Как решаем: ADMM + proximal operators

Главная вычислительная цена — SVD в обновлении low-rank компоненты

$$\mathcal{L}_\rho = \|L\|_* + \lambda \|S\|_1 + \langle Y, M - L - S \rangle + \frac{\rho}{2} \|M - L - S\|_F^2$$

$$L^{k+1} = \text{SVT}_{1/\rho}(M - S^k + Y^k/\rho)$$

$$S^{k+1} = \text{soft}_{\lambda/\rho}(M - L^{k+1} + Y^k/\rho)$$

$$Y^{k+1} = Y^k + \rho(M - L^{k+1} - S^{k+1})$$

$$\text{SVD}(p \times T): \quad \mathcal{O}(\min(pT^2, p^2T))$$

SVT для L

Singular Value Thresholding: SVD + shrinkage сингулярных чисел. Даёт low-rank фон.

Soft-thresholding для S

Пороговая обработка элементов матрицы. Даёт sparse foreground / выбросы.

Почему нужны быстрые варианты

При больших p и T полный SVD становится bottleneck; поэтому сравниваем GoDec, windowed GoDec и randomized PCA.

Какие методы сравниваем

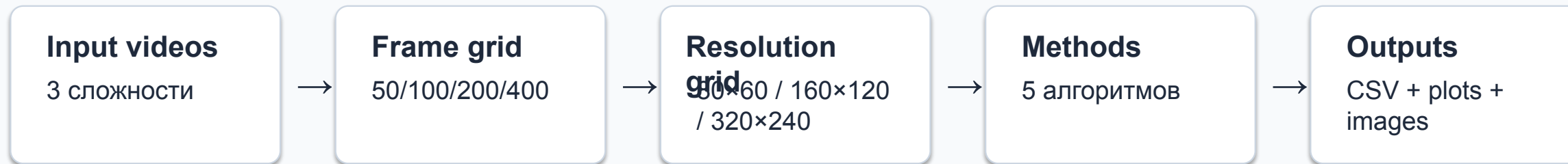
Роль метода, bottleneck и ожидаемое поведение

Метод	Роль в проекте	Bottleneck	Ожидаемое поведение
PCA	speed baseline, не robust	SVD / covariance	очень быстро, но $S = M - L$ не гарантирует foreground
Randomized PCA	самый быстрый baseline	randomized SVD	минимальный runtime, грубая low-rank аппроксимация
GoDec	approximate batch RPCA	low-rank approximation + thresholding	быстрее ADMM, residual выше
Windowed GoDec	block/scalable RPCA	разбиение по окнам	лучший practical compromise для видео
RPCA-ADMM	classical convex PCP baseline	SVD на итерациях	лучший residual среди robust-методов, самый тяжёлый

Ключевое сравнение: convex accuracy baseline (RPCA-ADMM) против speed/scale baselines и approximate RPCA.

Экспериментальный pipeline

Скрипт автоматически строит таблицы, графики и visual decomposition grids



Автоматизация

run_scalability.py запускает все методы на сетке videos × frames × resolutions, собирает метрики и сохраняет результаты в outputs/.

Файлы результата

outputs/tables/experiments.csv
outputs/plots/*.png
outputs/images/*.png
summary_by_method.csv

- Для visual grids сохраняются: Original / Low-rank background / Sparse foreground / Binary mask.
- Для масштабируемости отдельно варьируются число кадров и разрешение.
- Все результаты ниже построены по отчетной CSV и картинкам из финального запуска.

Дизайн экспериментов

3 видео, 5 методов и 180 успешных запусков

Видео

video_01_simple
video_02_medium
video_03_hard

Размеры

80×60
160×120
320×240

Кадры

50 200
100 400

Методы

PCA, RandPCA
GoDec, W-GoDec
RPCA-ADMM

$3 \times 3 \times 4 \times 5 = 180$ successful
runs

Метрики

- runtime_sec — время работы метода
- peak_memory_mb — пиковое потребление памяти
- relative decomposition residual — насколько хорошо $L + S$ восстанавливает M
- sparsity(S) — доля разреженной компоненты после пороговой обработки
- effective_rank(L) — эффективная сложность фона
- visual decomposition grid — качественная оценка разделения без ground truth masks

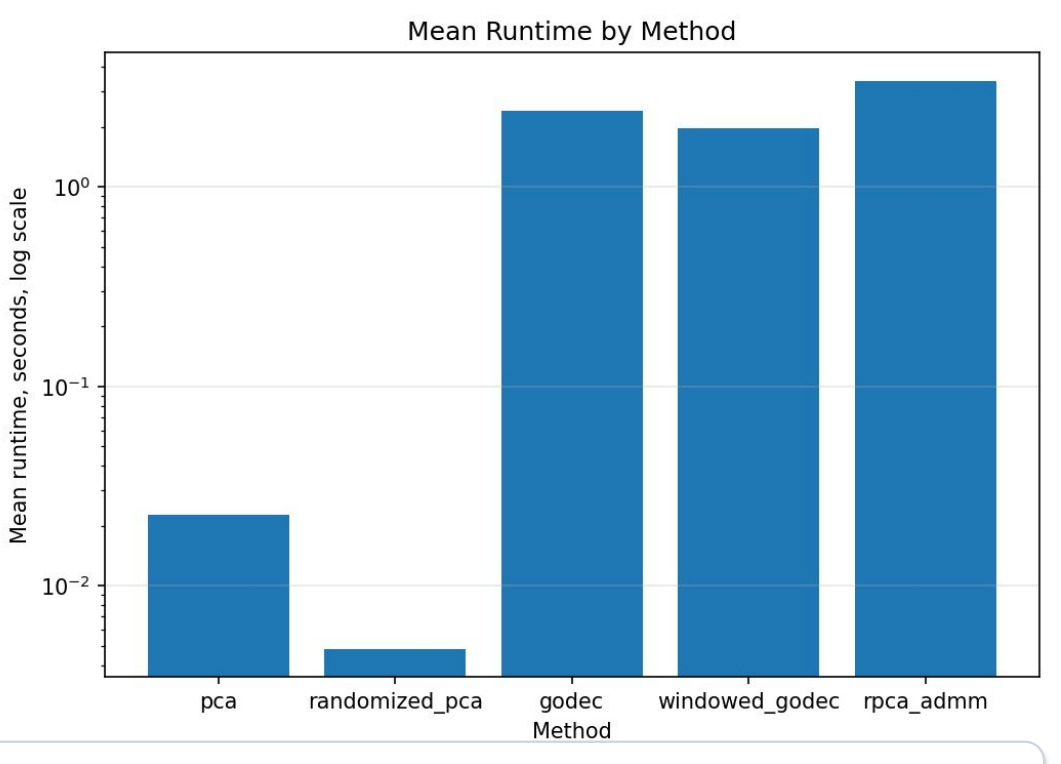
Почему нет F1/IoU

В финальном наборе нет ground truth foreground masks. Поэтому качество оцениваем через residual + визуальные grids; F1/IoU — естественное улучшение проекта.

Итоговые численные результаты

Средние значения по 180 запускам

method	runtime, sec	memory, MB	residual	rank
randomized_pca	0.0048	74.75	0.0000	5.00
pca	0.0227	107.85	0.0000	5.00
windowed_godec	1.9767	170.97	0.0192	9.17
godec	2.3948	211.26	0.0311	5.00
rpca_admm	3.4021	211.10	0.0085	13.86



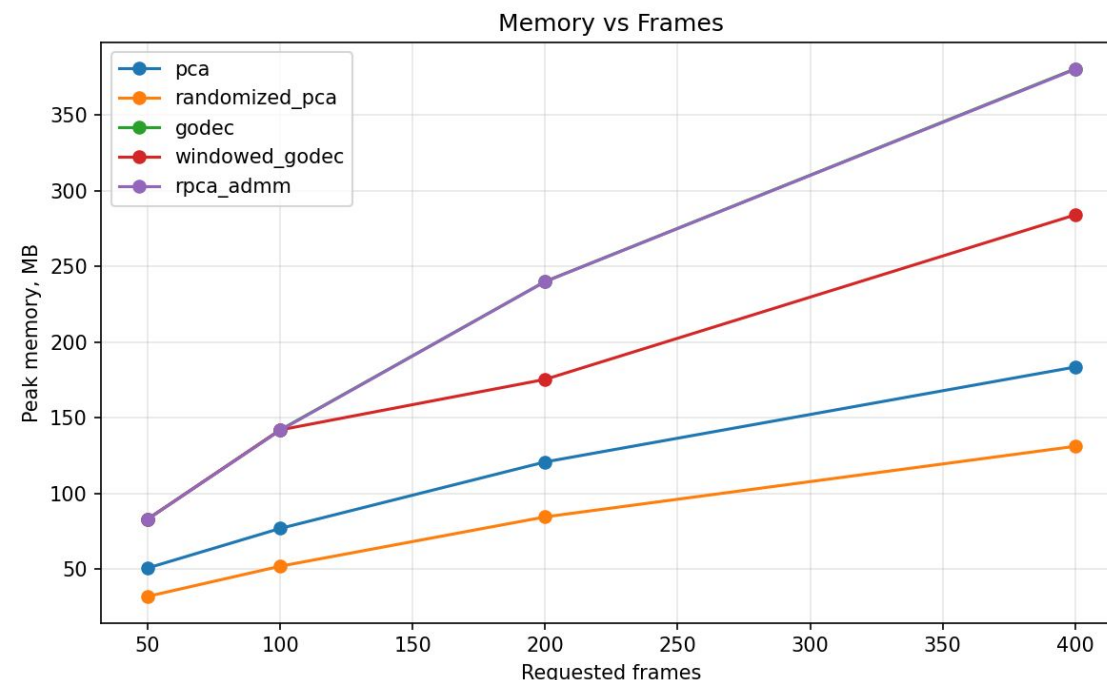
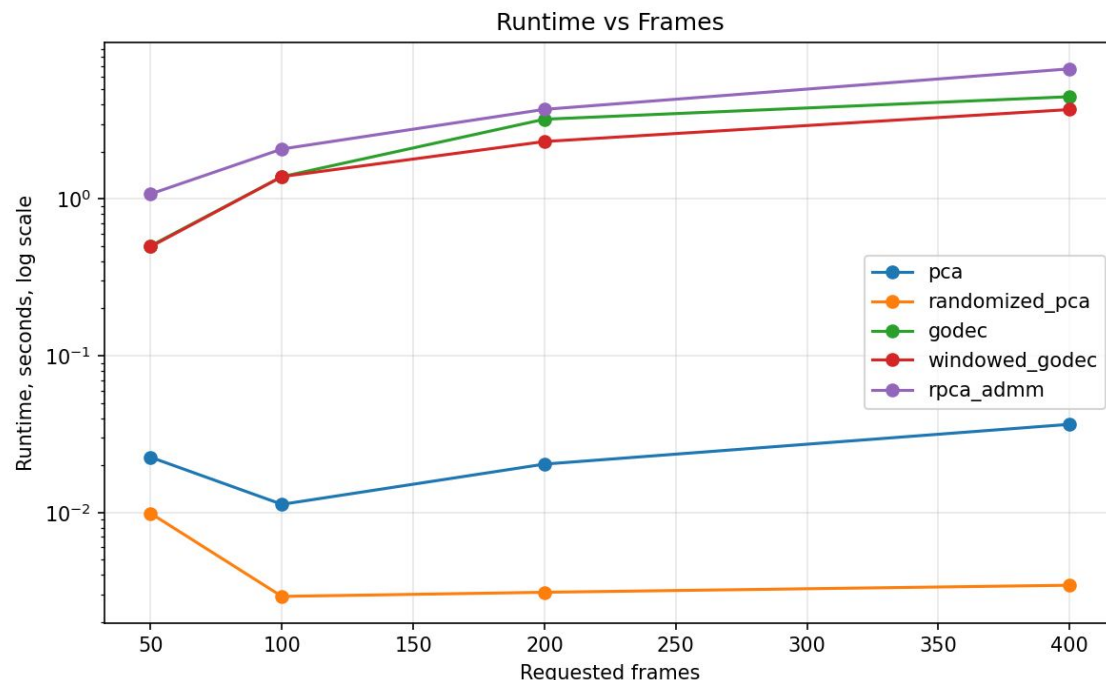
Интерпретация таблицы

Randomized PCA самый быстрый, но это не robust baseline. Среди robust-методов RPCA-ADMM имеет лучший residual (0.0085), однако самый большой runtime. Windowed GoDec быстрее batch GoDec и использует меньше памяти.

Важно: residual = 0 у PCA/randomized PCA не означает лучшую foreground-сегментацию, потому что там $S = M - L$ задаётся как остаток от обычной PCA, а не robust sparse-компонента.

Масштабируемость по числу кадров

При росте T увеличивается цена SVD-based методов

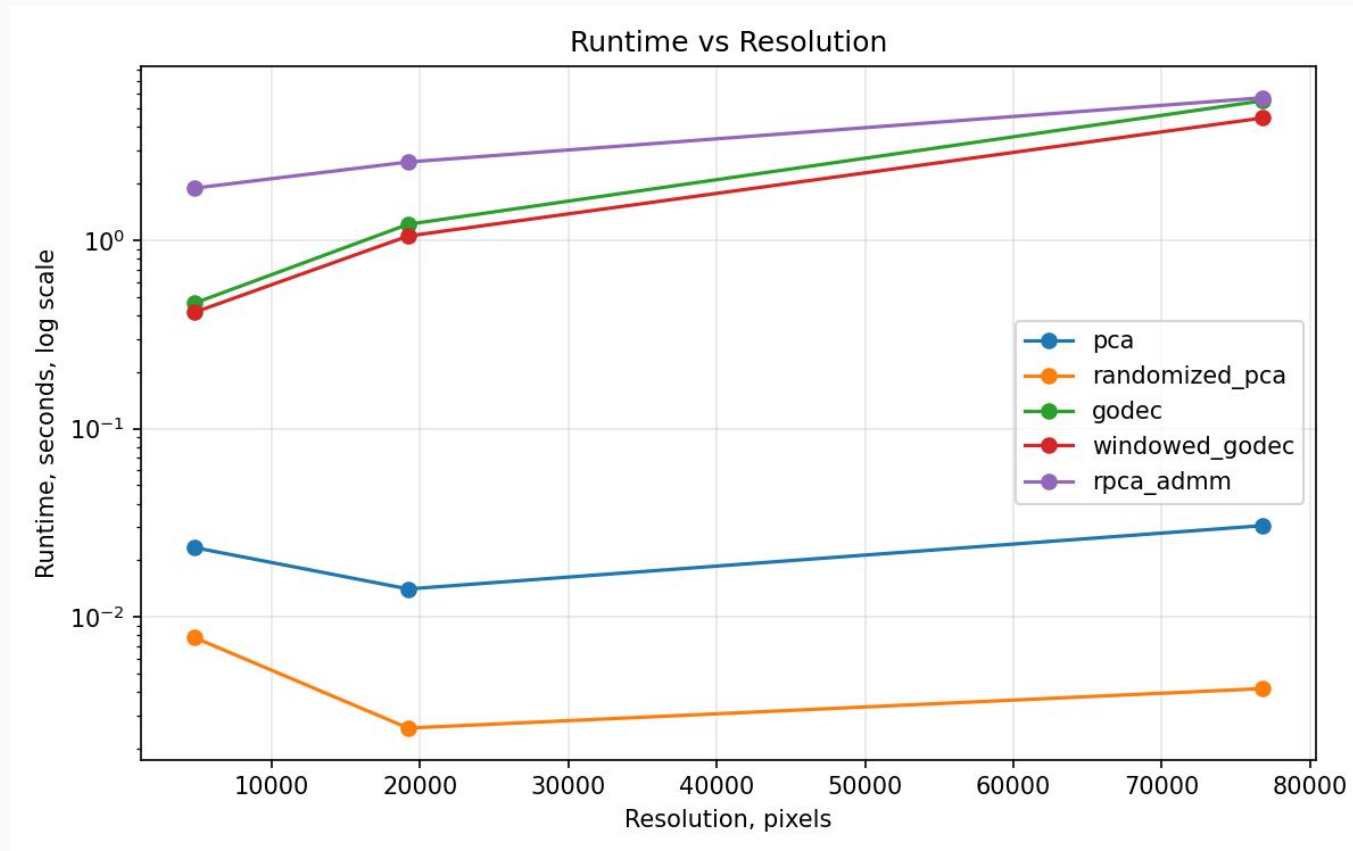


Вывод по frames

ADMM тяжелее по runtime, потому что делает SVD на итерациях. Windowed GoDec снижает стоимость относительно batch GoDec. На memory-графике GoDec может визуально сливаться с ADMM: в нашей реализации их peak memory близка.

Масштабируемость по разрешению

Рост $H \times W$ увеличивает число строк матрицы M



Что происходит

$p = H \cdot W$ растёт линейно с числом пикселей, но стоимость SVD растёт существенно быстрее.

Практический смысл

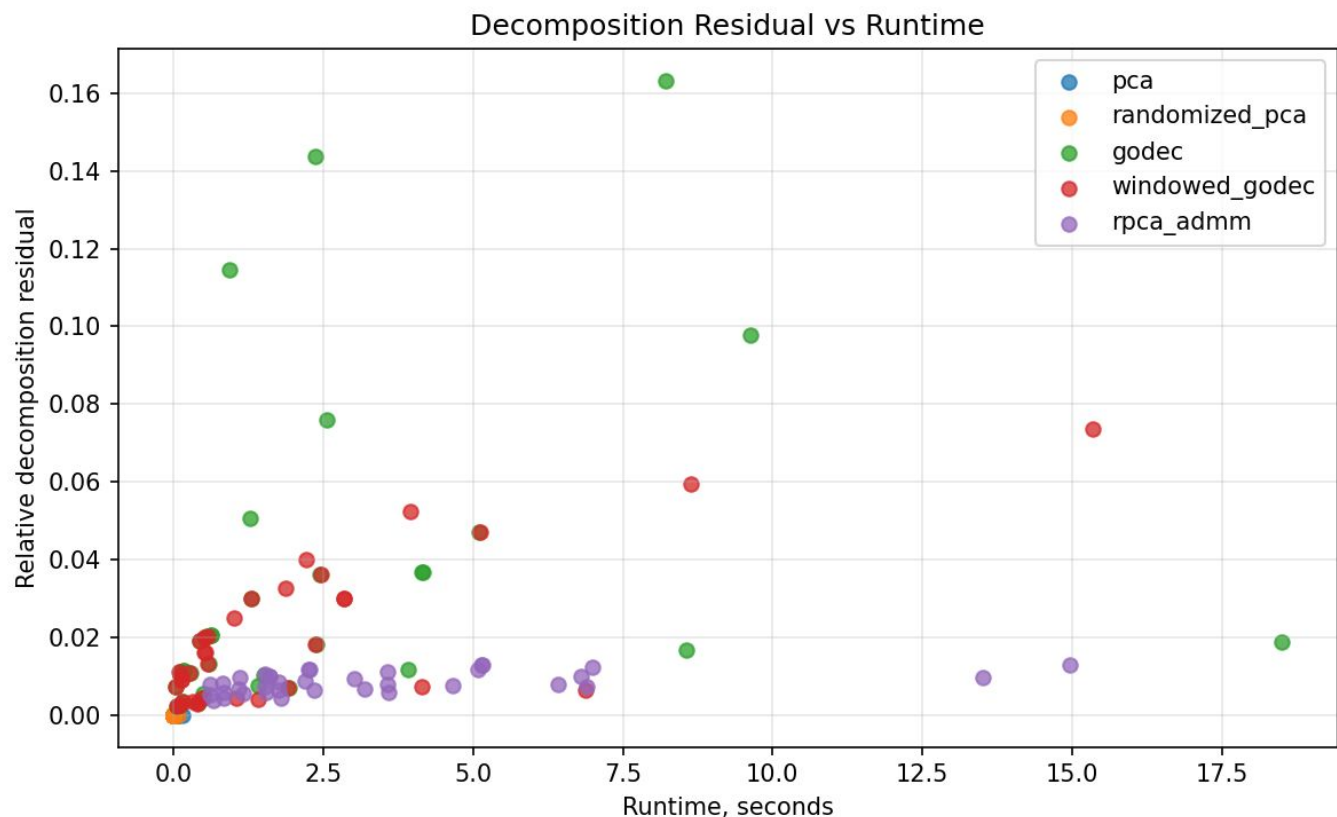
Для больших видео выгодны windowed/approximate методы или partial/randomized SVD внутри ADMM.

Итог

Классический convex baseline точный, но плохо масштабируется без оптимизаций SVD.

Decomposition residual vs runtime

Компромисс между точностью разложения и вычислительной ценой



ADMM

Минимальный residual среди robust-методов, но требует больше времени.

Windowed GoDec

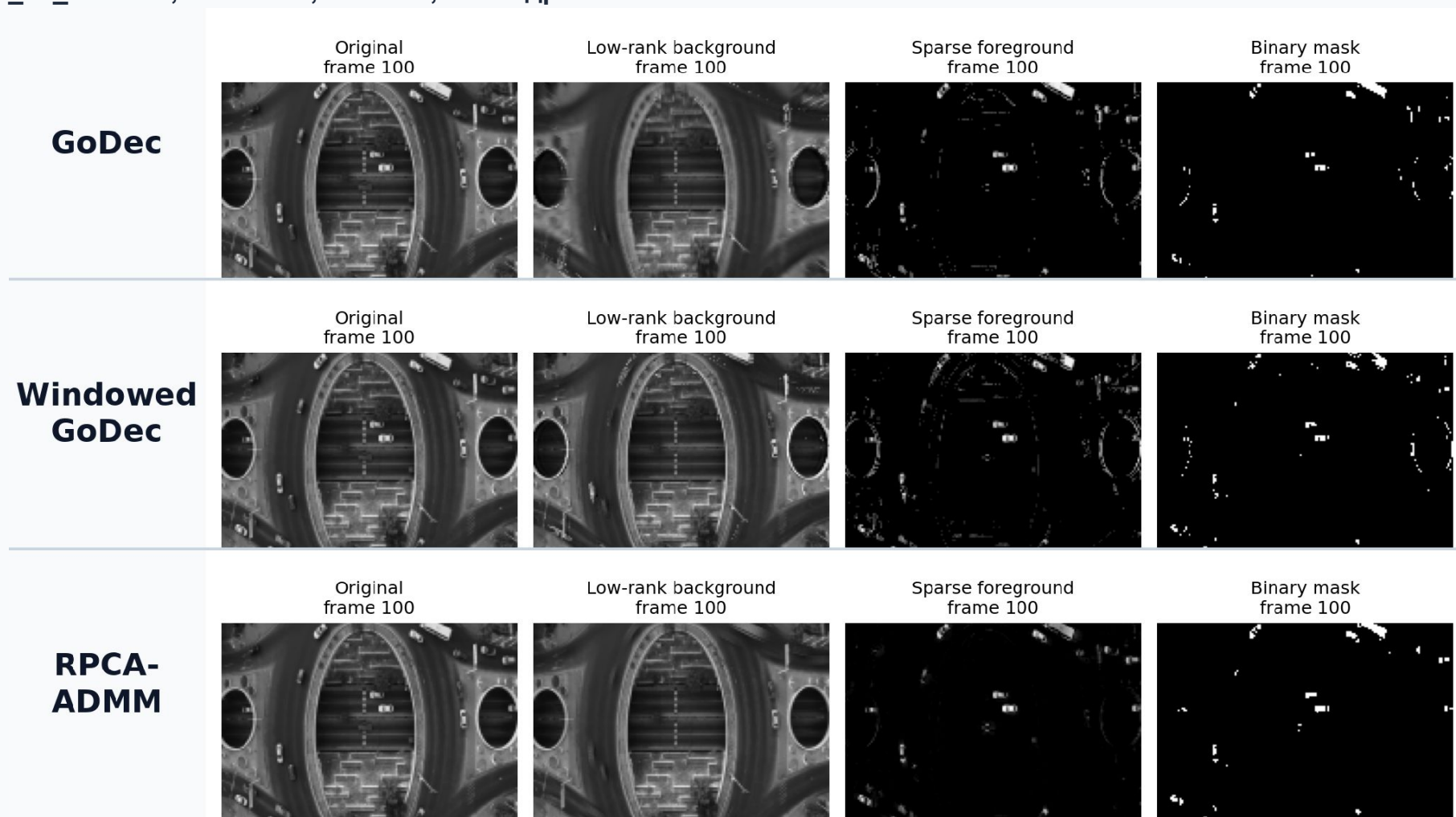
Более практичный компромисс, чем обычный GoDec: ниже runtime и memory при приемлемом residual.

Baseline PCA

Очень быстрый, но не решает robust PCP и может давать неинтерпретируемый foreground.

Визуальная часть результата

Пример: video_02_medium, frame 100, 160×120, 200 кадров



Сетка показывает Original / Low-rank background / Sparse foreground / Binary mask. PCA не показан здесь, потому что он baseline и не является robust-методом.

Ограничения модели и риски

RPCA хорошо работает при стационарной камере и относительно редком движении

Модельные ограничения

Динамический фон, тени, шум и движение камеры нарушают предположение low-rank + sparse.

Практический риск

Если объект долго находится в центре кадра, он может частично попасть в background.

Ограничение оценки

Без ground truth masks нельзя честно считать F1/IoU; residual не равен качеству segmentation.

Как улучшить

- Стабилизация камеры перед RPCA для динамичных сцен.
- Partial/randomized SVD внутри ADMM, чтобы снизить bottleneck.
- Online RPCA / GRASTA для потокового видео вместо batch-обработки.
- Постобработка foreground masks: morphology, connected components, adaptive threshold.
- Добавить ground truth masks и считать F1/IoU для объективной segmentation-оценки.

Подробный вывод по проекту

Что планировалось, что получилось и как развивать дальше

Что планировалось

- Реализовать RPCA для background subtraction.
- Сравнить методы на нескольких видео.
- Проверить масштабируемость по frames и resolution.
- Показать не только objective/runtime, но и recovered background/foreground.

Что получилось

- 5 методов: PCA, randomized PCA, GoDec, windowed GoDec, RPCA-ADMM.
- 3 видео и 180 успешных запусков.
- Таблицы, графики масштабируемости и visual grids.
- ADMM лучший по residual среди robust-методов; windowed GoDec — practical compromise.

Как улучшить

- Ground truth masks и F1/IoU.
- Online RPCA / GRASTA.
- Partial/randomized SVD в ADMM.
- Стабилизация камеры.
- Больше видео и более сложные сцены.

Главный итог: convex ADMM точнее, windowed/approximate методы практичнее для больших видео

Спасибо за внимание

Robust PCA / Principal Component Pursuit для background subtraction

GitHub проекта

<https://github.com/VotetoDaa/methopt-robust-pca>

Базовая статья

Candès, Li, Ma, Wright — Robust Principal Component Analysis? arXiv:0912.3599